

```
[
{
    "q": "How to do the ADF Test for each column of my Dataframe in Python? I am looking for a way to do the ADF Test for multiple columns of a DataFrame. Do you have any idea? This is my multi-column dataframe. How to calculate the test for each column separately?",
    "a": "from statsmodels.tsa.stattools import adfuller\nimport pandas as pd\n\n\ndef adfuller_test(series, signif=0.05):\n    \"\"\"\n    Perform Augmented Dickey-Fuller to test for Stationarity of the given series\n    and print report.\n    Null Hypothesis: Data has unit root and is non-stationary.\n    series: time series in pd.Series format\n    signif: significance level for P-value to reject Null Hypothesis\n    \"\"\"\n    x = adfuller(series, autolag='AIC')\n    #using dictionary saves different data types (float, int, boolean)\n    output = {'Test Statistic': x[0], \n              'P-value': x[1], \n              'Number of lags': x[2], \n              'Number of observations': x[3],\n              f'Reject (signif. level {signif})': x[1] < signif }\n    for key, val in x[4].items():\n        output[f'Critical value {key}'] = val\n    return pd.Series(output)\n"}
{
    "q": "Extracting City and State from a column in CSV using python. so, i'm trying to extract the city and state from the given data...",
    "a": "all_data = pd.DataFrame({'Purchase Address': ['917 1st St, Dallas, TX 75001',\n                                                    '682 Chestnut St, Boston, MA 02215',\n                                                    '669 Spruce St, Los Angeles, CA 90001',\n                                                    '669 Spruce St, Los Angeles, CA 90001']})\n\ndef get_city_state(address):\n    asplit = address.split(',')\n    ssplit = address.split(' ')\n    city = asplit[1].split()[-1]\n    state = asplit[2].split()[0]\n    return city\n\nall_data['City'] = all_data['Purchase Address'].apply(get_city_state).to_list()\n\ndef get_city_state(address):\n    asplit = address.split(',')\n    ssplit = address.split(' ')\n    city = asplit[1].split()[-1]\n    state = asplit[2].split()[0]\n    return states\n\nall_data['States'] = all_data['Purchase Address'].apply(get_city_state).to_list()\n\ndef get_city_state(address):\n    asplit = address.split(',')\n    ssplit = address.split(' ')\n    city = asplit[1].split()[-1]\n    state = asplit[2].split()[0]\n    return city , state\n\nall_data[['City', 'State']] = all_data['Purchase Address'].apply(get_city_state).to_list()\n"}
{
    "q": "How to create non-alphabetically ordered Categorical column in Polars Dataframe? [duplicate]. In Pandas, you can create an \"ordered\" Categorical column from existing string column as follows: In Polars documentation, I couldn't find such way to create ordered columns. However, I could reproduce this by using pl.from_pandas(df) so I suspect that this is possible with Polars as well. What would be the recommended way to this? I tried to create new column with: but I don't know how to include the custom ordering to this. I also checked In polars, can I create a categorical type with levels myself?, but I would prefer not to add another column to my Dataframe only for ordering.",
    "a": "my_order = [\"k\", \"z\", \"b\", \"a\"]\n\nndf.with_columns(pl.col(\"cats\").cast(pl.Enum(my_order)))\n\n(df.with_columns(pl.col(\"cats\").cast(pl.Enum(my_order)))\n   .sort(\"cats\")\n   .with_columns(order = pl.col(\"cats\").to_physical()))\n\n}\n"}
{
    "q": "pandas calculate delta time. Here's some code where that will generate some random data, and chart plus lines representing 30th & 90th percentiles. Outputs: (minus the highlighted part of the chart) Im trying to figure out if its possible to calculate the time in the data it takes to reach (highlighted yellow) from green to the red line. I can manually enter in the data: Which will output: But if I attempt to use: This will throw an ValueError: Can only compare identically-labeled Series objects. Would there be a better method to madness? Ideally it would be nice to create some sort of an algorithm that can calculate
```

delta Time to it takes to go from 30th - 90th percentile and then delta back from 90th - 30th.. But I may have to put some thought towards how that could be accomplished..",

```
"a": "minStart = df.loc[df['Random_Number'] < df.quantile(0.3)
[0]].index[0]\n\nmaxStart = df.loc[df['Random_Number'] > df.quantile(0.90)
[0]].index[0]\n\nhours = maxStart - minStart\nhours\n",
},
{
    "q": "Color Prediction System with OpenCv Python. Hi I am an initial stage
developer trying to make a color prediction system, but I came up with some
error that I can not understand. I think you could help me with this .... I will
share with you my proper code. This is the error: IndexError: index 1 is out of
bounds for axis 0 with size 1. The error showing in RGB2HEX(color): function but
I am unable to solve it. Here is the Code: ",
    "a": "from sklearn.cluster import KMeans\nimport matplotlib.pyplot as
plt\nimport numpy as np\nimport cv2\nfrom collections import Counter\nimport
os\nimport argparse\n\nndef RGB2HEX(color):\n    return \"#{:02x}{:02x}{:02x}
\".format(int(color[0]), int(color[1]), int(color[2]))\n\nndef
get_image(image_path):\n    image = cv2.imread(image_path)\n    image =
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)\n    # example to read as grayscale
image\n    # image = cv2.imread(image_path, 0)\n    return
image\n\nIMAGE_DIRECTORY = './path/to/images/'\n\nCOLORS = {\n    'GREEN': [0,
128, 0],\n    'BLUE': [0, 0, 128],\n    'YELLOW': [255, 255, 0]\n}\n\nimages = []
\n\nfor file in os.listdir(IMAGE_DIRECTORY):\n    if not file.startswith('.'):\n
images.append(get_image(os.path.join(IMAGE_DIRECTORY, file)))\n\n# extracting
colors from image\n\nndef get_colors(images, number_of_colors, show_char = True):
\n    for j in range(len(images)):\n        modified_image =
cv2.resize(images[j], (600, 400), interpolation = cv2.INTER_AREA)\n
modified_image =
modified_image.reshape(modified_image.shape[0]*modified_image.shape[1], 3)\n
# example to use with grayscale images\n        # modified_image =
modified_image.reshape(modified_image.shape[0]*modified_image.shape[1], 1)\n\n
clf = KMeans(n_clusters = number_of_colors)\n        labels =
clf.fit_predict(modified_image)\n\n        counts = Counter(labels)\n\n
center_colors = clf.cluster_centers_\n        # We get ordered colors by
iterating through the keys\n        ordered_colors = [center_colors[i] for i in
counts.keys()]\n        hex_colors = [RGB2HEX(ordered_colors[i]) for i in
counts.keys()]\n        rgb_colors = [ordered_colors[i] for i in counts.keys()]\n
\n\n",
    },
{
    "q": "Pandas Compare two data frames and look for duplicate elements. I want
to compare df and df_equal. df contains several individual data frames. how can
i check df for duplicate? I tried .equals: my desired output:",
    "a": "for row in range(len(df)):\n    print((df.iloc[row, ].values ==
df2.values).all())\n\n",
    },
{
    "q": "Load pickled pandas object (or any object) from file and return
unpickled object.",
    "a": "original_df = pd.DataFrame(\n    {\"foo\": range(5), \"bar\": range(5,
10)}\n)\n\noriginal_df\npd.to_pickle(original_df, \"./dummy.pkl\")\n",
    },
{
    "q": "Pickle (serialize) object to file.",
    "a": "original_df = pd.DataFrame(\n    {\"foo\": range(5), \"bar\": range(5,
10)}\n)\n\noriginal_df\noriginal_df.to_pickle(\"./dummy.pkl\")\n",
    },
{
    "q": "Read general delimited file into DataFrame.",
    "a": "pd.read_table(\"data.csv\")\n",
    },
{
    "q": "Read a comma-separated values (csv) file into DataFrame.",
```

```
    "a": "pd.read_csv(\"data.csv\")\n"  
  }  
]
```